
OpenCode RuneSmith Implementation Plan

refactorartist (Khalid Zubair)

2026-06-14

1 Motivation

RunicEngines is a new cooperative that will maintain multiple code repositories as it grows. Developers across all repositories need consistent access to AI-powered agents and skills for common development workflows: issue decomposition, code implementation, review, testing, documentation, and deployment.

Currently there is no shared automation infrastructure. Without a plugin, each repository would independently define its own agents and skills in `.opencode/agents/` and `.opencode/skills/`, duplicating effort and fragmenting conventions. This fragmentation undermines the cooperative's shared standards (ADR 0002) and creates drift that compounds over time.

The `@runicengines/opencode-runesmith` plugin establishes shared automation from day one. A single plugin declaration in `opencode.json` installs a complete suite of agents, skills, and startup infrastructure across all RunicEngines repos. This prevents fragmentation before it begins.

It is important to distinguish RuneSmith from the knowledge-base-specific agents defined in ADR 0005. Those agents (kb-architect, kb-editor, kb-tech-lead, etc.) automate content-creation workflows within the knowledge-base repository itself — writing ideas, knowledge notes, research documents, proposals, and ADRs. RuneSmith serves code repositories: it automates the issue-to-PR pipeline, enforces code review conventions, manages changelogs, and orchestrates CI/CD workflows. The two systems are separate by design (ADR 0007 §7).

Installation is a single line in the consumer project's `opencode.json`:

```
1 {  
2   "plugin": ["@runicengines/opencode-runesmith"]  
3 }
```

References: ADR 0007. See Appendix A for details on the plugin system and Appendix B for the agent architecture that this proposal builds upon.

2 Plugin Package Structure

Distribution: The plugin is distributed as a private npm package hosted on GitHub Packages under the `@runicengines` scope, per ADR 0007 §1. This provides versioning (SemVer), access control (org members only), and auto-install (Bun downloads and caches on startup).

Repository: The plugin lives in its own repository at github.com/RunicEngines/opencode-runesmith, separate from the knowledge-base. This keeps the release cycle, issue tracking, and CI/CD independent from KB content changes (ADR 0007 §6).

Package directory layout:

```
1 @runicengines/opencode-runesmith/
2 +-- dist/
3 |   +-- index.js           # Compiled init hook factory
4 +-- .opencode/
5 |   +-- agents/           # 8 agent .md files (rs-architect, rs-
6 |       +-- skills/       # 20 skill SKILL.md files (rs-issue-to-
7 |           +-- plan/, etc.)
8 +-- docs/
9 |   +-- migrations/       # Migration guides for version bumps
10 +-- package.json
11 +-- tsconfig.json
12 +-- README.md
```

The `.opencode/` directory mirrors the project-level structure. This is the directory the init hook copies from.

package.json conventions:

- `"type": "module"` — ESM throughout
- `"dependencies": { "@opencode-ai/plugin": "^..." }` — OpenCode plugin SDK
- `"publishConfig": { "registry": "https://npm.pkg.github.com" }` — GitHub Packages target
- `"main": "dist/index.js"` — Compiled init hook entry point
- Scripts for `build`, `prepublishOnly`, `test`

Build and publish workflow:

1. TypeScript source at `src/index.ts` compiles to `dist/index.js` via `tsc`
2. Tag push with `v*` pattern triggers GitHub Actions workflow
3. CI runs `npm test`, then `npm publish`
4. Release notes auto-generated from Conventional Commits between tags

Consumer setup:

Each developer configures per-repo or global `.npmrc`:

```
1 @runicengines:registry=https://npm.pkg.github.com/
```

And sets a `GITHUB_TOKEN` environment variable with `read:packages` scope. The plugin is declared in `opencode.json`:

```
1 {  
2   "plugin": ["@runicengines/opencode-runesmith"]  
3 }
```

Bun auto-installs on first startup, caching in `~/ .cache/opencode/node_modules/`.

Versioning (SemVer 2.0.0) per ADR 0007 §1:

Bump	Trigger
Major	Breaking changes to agent file structure, init hook contract, or activation API
Minor	New agents, skills, MCP support, non-breaking additions
Patch	Bug fixes, prompt tweaks, dependency bumps

Multi-version coexistence: The organisation standardises on a single major version for all repos. If a repo falls behind, remediation is a cache clear followed by restart:

```
1 bun cache rm @runicengines/opencode-runesmith && bun install
```

See Appendix A for details on the OpenCode plugin system.

3 Init Hook and Version-Stamping

The central constraint: OpenCode does not auto-discover bundled `.opencode/` directories inside npm plugins. It scans only the project worktree and the global config directory (`~/ .config/ opencode/`). Therefore, agent and skill files bundled in the npm package are invisible to OpenCode unless copied into the project's `.opencode/` tree.

Solution: The plugin exports a factory function that copies agents and skills from the npm package into the project's `.opencode/ {agents, skills} /` directories at every startup. This is an event hook registered via the plugin SDK.

The factory runs on EVERY OpenCode startup. It is not a one-time install script — it must be idempotent and fast on normal startups.

Three operating modes controlled by version-stamping:

Condition	Action	Performance
No stamp file (first install)	Full copy of all agent <code>.md</code> and skill <code>SKILL.md</code> files; write stamp	Slow (one-time)
Stamp mismatch (update)	Clear old files, re-copy from updated package; update stamp	Moderate
Stamp matches (normal)	No file operations, return immediately	Fast (<10ms)

Version-stamping mechanism:

A plain text file at `.opencode/ .runesmith-version` contains a semver string (e.g., `1.0.0`). The init hook reads this file on startup, compares it against the installed plugin's `package.json` version, and selects the operating mode accordingly.

```
1 .opencode/  
2 +-- .runesmith-version # "1.0.0"  
3 +-- agents/  
4 |   +-- rs-architect.md  
5 |   +-- rs-spec-writer.md  
6 |   +-- ...  
7 +-- skills/  
8     +-- rs-issue-to-plan/SKILL.md  
9     +-- rs-pr-packager/SKILL.md  
10    +-- ...
```

Error handling — fail-open strategy:

Copy failures log warnings with a [`runesmith`] prefix but do NOT prevent event hooks from loading. A partial copy leaves the project with stale agents/skills (from the previous successful startup), and the warning alerts the developer to re-run or investigate. The plugin's event hooks (if any) continue to register.

Update workflow for consumers:

1. Plugin maintainer publishes `v1.1.0` to GitHub Packages
2. Consumer runs `bun cache rm @run engines/opencode-runesmith` to clear cached old version
3. Consumer restarts OpenCode — Bun downloads new version
4. Init hook detects version mismatch (stamp says `1.0.0`, package says `1.1.0`)
5. Clear old files, copy new files, update stamp to `1.1.0`
6. OpenCode continues startup — new agents and skills are available

See Appendix A for details on the OpenCode plugin system (init hook lifecycle and bundling).

4 Agent Architecture

RuneSmith uses a hub-and-spoke orchestration model per ADR 0007 §2. The **Architect** is the sole orchestrator — the only agent with `task` permission — and coordinates work across six specialist leaf agents. Leaf agents have `task: deny`, preventing delegation loops.

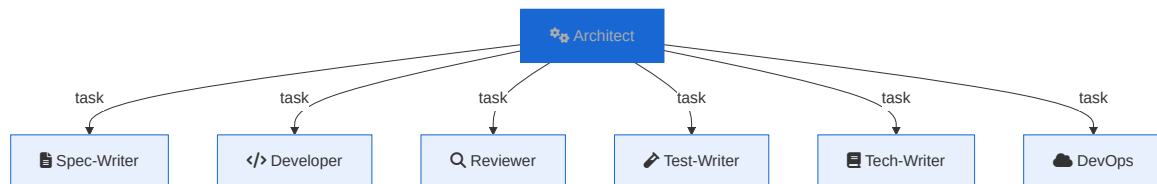


Figure 1: Hub-and-Spoke Architecture

Phase gates: Plan → Implement → Review → Test → Deploy, with circuit breakers at each gate.

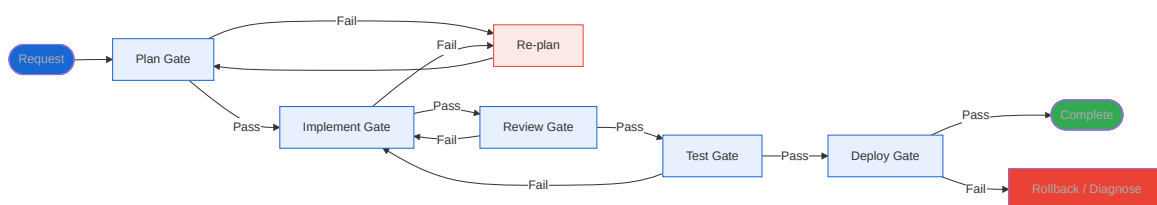


Figure 2: Phase Gate Pipeline

If a gate fails, the architect retries up to 3 times. After 3 failures, it re-plans the approach. If re-planning also fails, it escalates to the user.

Error recovery ladder: Retry (up to 3) → Re-plan → Escalate to user.

Per-agent configuration (ADR 0007 §9):

Agent	Model	Temp	Bash	Edit	Task	Key Re-sponsibility
Architect	v4-pro	0.1	git/gh only	allow	allow (6 specialists)	Orchestration + gate validation

Agent	Model	Temp	Bash	Edit	Task	Key Re-sponsibility
Spec-Writer	v4-flash	0.2	gh only	allow	deny	Issue to structured spec
Developer	v4-pro	0.2	*:ask	allow	deny	Code implementation
Reviewer	v4-flash	0.0	git diff/log	deny	deny	Code review (S1–S5)
Test-Writer	v4-flash	0.2	test cmds	allow	deny	Tests + coverage
Tech-Writer	v4-flash	0.3	deny	allow	deny	Documentation
DevOps	v4-pro	0.1	docker/CI	allow	deny	CI/CD + deploy

Naming convention: All agents use the `rs-` prefix to prevent namespace collisions with KB agents (`kb-*`) or user-defined agents. Agent files are named `rs-{role}.md` (e.g., `rs-architect.md`, `rs-developer.md`).

Discovery: Agent files are copied to the project's `.opencode/agents/` directory by the init hook (Section 3). OpenCode discovers them from there automatically — no additional registration step.

Agent file format follows the specification described in Appendix B. Each file defines model selection, temperature, tool permissions, system prompt, and optional subagent declarations.

See Appendix B for agent system reference and Appendix D for orchestration patterns.

5 Skill Architecture

Skills are reusable instruction bundles loaded on demand via `skill({ name: "rs-{name}" })`. They are NOT injected into every conversation — skills are loaded only when a user prompt or chained invocation triggers them. This keeps context windows small and agent prompts focused.

Skill naming: The `rs-` prefix matches the agent namespace convention. Skill names follow `^[a-z0-9]+(-[a-z0-9]+)*$` per the skill file format described in Appendix C.

Trigger types:

Type	Behaviour
Manual	Invoked by user prompt (e.g., “run rs-pr-packager”)
Automatic	Triggered by webhook bridge (e.g., incoming GitHub webhook)
Chained	Invoked by another skill or agent during a multi-step workflow

20 skills across 3 categories (ADR 0007 §10):

5.1 Workflow skills (8)

Skill	Trigger	Description
<code>rs-issue-to-plan</code>	Manual + chained	Decompose GitHub issue into phased implementation plan with requirements, files-to-change, acceptance criteria, and test strategy. Follows ADR 0002 conventions for branch naming and commit messages.

Skill	Trigger	Description
<code>rs-pr-packager</code>	Manual	Generate PR descriptions from local commits conforming to ADR 0002. Validates branch names <code>{type}/{issue-number}-{kebab-description}</code>), enforces Conventional Commits, links issues. Optionally pipes output into <code>gh pr create</code> .
<code>rs-changelog-manager</code>	Manual	Maintain CHANGELOG.md per Keep a Changelog 2.0.0. Manages the Unreleased section, promotes to a versioned release on cut, yanks releases as needed.
<code>rs-test-helper-run</code>	Chained	Run test suites with configurable framework (pytest, vitest, etc.) and collect structured results.
<code>rs-test-helper-diagnose</code>	Chained	Analyse test failures, classify by type (code bug, test bug, flaky infrastructure), suggest root causes and fixes.
<code>rs-doc-architect</code>	Manual	Plan documentation structure per the Diataxis framework — audits existing docs, classifies into quadrants, identifies gaps.
<code>rs-commit-writer</code>	Manual	Generate Conventional Commit messages from staged diff. Maps commit types to Keep a Changelog 2.0.0 sections.

Skill	Trigger	Description
<code>rs-pr-writer</code>	Manual	Generate PR body from issue and spec for draft PRs before coding begins. Complements <code>rs-pr-packager</code> .

5.2 Review skills (6)

Skill	Trigger	Description
<code>rs-review-methodology</code>	Chained	Structured review process per ADR 0002 §5 with checklist-based evaluation and pass/fail gates.
<code>rs-review-severity</code>	Chained	Classify findings S1 (critical) through S5 (informational) with defined response SLAs — S1 blocks merge, S2 requires fix before merge, S3 documented and tracked, S4 discretionary, S5 noted.
<code>rs-review-security</code>	Chained	Security-specific patterns: credential scanning, injection vulnerabilities, authentication flaws, dependency risks.
<code>rs-review-code</code>	Chained	Code review following Google Engineering Practices: design, functionality, complexity, tests, naming, comments, style, consistency, documentation. Uses Nit/Optional/FYI severity labelling.

Skill	Trigger	Description
<code>rs-review-architecture</code>	Chained	Architecture review following the Azure Well-Architected Framework pillars (Reliability, Security, Cost Optimisation, Operational Excellence, Performance Efficiency) combined with the C4 model (System Context, Container, Component, Code) for visualisation.
<code>rs-doc-auditor</code>	Chained	Audit documentation for structure, completeness, and compliance against Diataxis quadrants. Produces a per-quadrant compliance score.

5.3 Utility skills (6)

Skill	Trigger	Description
<code>rs-discover</code>	Chained	Codebase scanning: identify entry points, module structure, test layout, coding conventions, dependency manifests, CI configuration. Used for initial orientation in unfamiliar repos.
<code>rs-consult</code>	Chained	SME consultation: load domain-specific knowledge from embedded references for technology choices, design patterns, and architectural conventions.

Skill	Trigger	Description
<code>rs-scratchpad</code>	Chained	Session scratchpad lifecycle: init (creates date-branch directory under <code>.runesmith/scratchpad/</code>), clear (removes session artifacts), status (lists active scratchpads).
<code>rs-dependency-checker</code>	Chained	Scan dependencies for known vulnerabilities (CVE lookup), outdated packages (compared to latest Semver-compatible), and licence compliance violations.
<code>rs-doc-llm-txt</code>	Manual	Generate <code>/llms.txt</code> and expanded context files (<code>llms-ctx.txt</code> , <code>llms-ctx-full.txt</code>) from project documentation per the <code>llms.txt</code> standard.
<code>rs-env-validator</code>	Chained	Validate <code>.env</code> against <code>.env.example</code> for missing required variables, malformed values, and format violations.

Hardcoded routing: Skills are listed directly in agent prompts rather than via a dynamic routing YAML file. At this scale (20 skills, 8 agents), the indirection of a routing layer adds complexity without benefit. Each agent's system prompt explicitly enumerates which `rs-*` skills it may invoke.

Skill file format: Each skill follows the `SKILL.md` format: frontmatter (name, description, trigger type, tool requirements) plus body (purpose, workflow steps, input/output contract). See Appendix C for the full skill system reference.

See Appendix C for the skill system reference.

6 KB Discovery

The knowledge-base is searchable but NOT delegatable per ADR 0007 §7. Plugin agents query KB content via the `rs-discover` skill but CANNOT invoke KB agents. Conversely, KB agents cannot invoke RuneSmith agents.

Discovery mechanism:

The `rs-discover` skill locates the KB repository from any code repo by checking common sibling directory conventions:

1. Check if `./knowledge-base/` exists relative to the consumer repo root
2. If running inside the knowledge-base repo itself, use the local `knowledge/` directory
3. If neither exists, KB access is unavailable — the skill scopes its search to the consumer repo only

Search capabilities when KB is found:

Operation	Description
Full-text search	Grep across all <code>.md</code> files in <code>knowledge/</code>
Cross-reference lookup	Trace <code>sources</code> , <code>references</code> , <code>related_*</code> links between documents
Pipeline tracing	Follow the content pipeline: <code>idea</code> → <code>knowledge</code> → <code>research</code> → <code>proposal</code> → <code>ADR</code>

Separation boundary: KB and RuneSmith are independent systems. The `rs-discover` skill provides read-only access to KB content. Plugin agents never write to the KB, invoke KB agents, or depend on KB agents for their core workflow.

7 MCP Integration

Current limitation: The OpenCode plugin SDK does not support programmatic MCP server registration (ADR 0007 §12). Plugins cannot register MCP servers at runtime through the SDK API.

Approach:

The plugin ships configuration snippets in its README for manual setup. Consumers add MCP server declarations under the `mcp` key in their `opencode.json`:

```
1 {
2   "mcp": {
3     "servers": {
4       "my-server": {
5         "type": "local",
6         "command": "npx",
7         "args": ["@modelcontextprotocol/server-filesystem", "."]
8       }
9     }
10  }
11 }
```

MCP servers can be configured as: - **Local subprocesses** — spawned as child processes by OpenCode, communicating via stdio - **Remote HTTP endpoints** — accessed via SSE or WebSocket for shared services

Context window consideration: MCP servers add tokens to the context window with each tool call. Only enable servers that agents actually need. The README includes guidance on which MCP servers are recommended for each agent role.

Future-proofing: If the SDK adds programmatic MCP registration, the init hook can be extended to auto-configure MCP servers from a plugin-defined manifest. This avoids breaking existing manual configurations.

See Appendix E for the MCP reference.

8 Permission Model

RuneSmith adopts OpenCode's three-tier permission model: `allow` (runs without approval), `ask` (prompts user), `deny` (blocks). Permissions are configured per-agent in the agent `.md` file.

Bash command granularity: Uses OpenCode's last-match-wins resolution. Start with `"*": "deny"` then add specific allowed patterns:

```
1 bash:
2   "*": deny
3   "git *": allow
4   "gh *": allow
```

Key restrictions per ADR 0007 §9:

Agent	Restriction	Rationale
All leaf agents	<code>task: deny</code>	Prevent delegation loops — only the Architect can delegate
Reviewer	<code>edit: deny</code>	Read-only reviewer — cannot modify code under review
Tech-Writer	<code>bash: deny</code>	No shell access — outputs only markdown documents
Developer	<code>bash: { "*": "ask" }</code>	All commands require approval; destructive commands explicitly denied in sub-patterns
DevOps	<code>kubectl *: deny,</code> <code>aws *: deny</code>	Infrastructure must be managed through CI/CD pipelines, not ad-hoc agent shell access

Enforcement at agent file level: Permission profiles are encoded in each agent's `.md` file — not gated by plugin version. This means permission changes take effect with the next init hook copy (or immediately if the agent is reloaded).

See Appendix B for the agent system reference, including the permission model.

9 Rollout Strategy

Immediate org-wide adoption per ADR 0007 §13: The plugin is rolled out to all RunicEngines repositories simultaneously, without a phased pilot or canary release.

Rationale: RunicEngines is a new cooperative without established workflows. There is no existing automation to migrate from. The plugin establishes the foundation rather than displacing an existing setup. Immediate rollout avoids the overhead of maintaining parallel systems during a phased transition.

Feature gating: Controlled through two mechanisms: - **Plugin version pinning** — repos pin to a specific version in their CI/CD configuration. If a breaking change is published, repos upgrade on their own schedule. - **Per-repo configuration** — individual repos can override agent permissions or disable specific skills through their local `.opencode/` overrides (OpenCode merges project-level config on top of plugin-provided config).

Rollback criteria and procedures:

Severity	Definition	Action
S1	Agent produces harmful output (deletes files, exposes secrets)	Immediate rollback — pin to previous version, file security report
S2	Agent workflow completely broken for common tasks	Rollback within 4 hours, file bug report
S3	Agent workflow degraded for edge cases	Document as known issue, fix in next patch
S4	Minor cosmetic or UX issues	Fix in next minor release
S5	Informational — no functional impact	Track in backlog

Rollback is executed by reverting the `opencode.json` plugin version (or tag if pinned via GitHub Packages) and clearing the Bun cache.

Feedback collection:

- **Surveys:** Quarterly developer experience surveys targeting workflow satisfaction, agent accuracy, and friction points.

- **Telemetry:** Opt-in telemetry stored in `.runesmith/telemetry/` directory — agent invocation counts, error rates, gate failure rates. No code content or personal data is collected. Opt-in is configured in `opencode.json` via a `runesmith.telemetry` flag.

10 Maintenance and Governance

Maintainers:

Role	Person
Core maintainer	refactorartist (Khalid Zubair)
Backup maintainer	RunicEngines members (collective)

The backup maintainer role is collective — any org member can step in if the core maintainer is unavailable, with the expectation that the change is reviewed by at least one other org member before release.

Contribution pipeline:

Stage	Path
Research First (in Knowledge Base)	Idea → Knowledge → Research → Proposal
Implementation (in plugin repo)	PR in github.com/RunicEngines/opencode-runesmith
Bug fixes	Direct PR in plugin repo
Prompt refinements	Direct PR in plugin repo
Dependency bumps	Direct PR in plugin repo

Non-trivial additions (new agents, new skills, architecture changes) must first go through the full Research First pipeline and produce a proposal in the Knowledge Base before any implementation PR is opened in the plugin repo. This ensures alignment with org-wide conventions and that the research exists before code is written.

Deprecation policy:

- **Notice period:** One minor release — when a feature is deprecated in version `x.y.z`, it is removed in version `x.y+1.z` at the earliest.
- **Migration guides:** Published under [docs/migrations/](#) in the plugin repository, covering the old and new approach, timing, and any manual steps.
- **Changelog marking:** Deprecated features are marked with `**Deprecated:` prefix in the CHANGELOG.md entry.

Release cadence per ADR 0007 §1:

Type	Cadence
Patch	As needed (bug fixes, prompt tweaks)
Minor	Monthly (new agents, skills, capabilities)
Major	At most one per quarter (breaking changes)

Pre-release versions: Use `-alpha`, `-beta`, `-rc` suffixes with npm `next` dist-tag for testing before stable release:

```
1 npm publish --tag next
```

GitHub Actions automation: On push of a `v*` tag, the release workflow compiles TypeScript, runs tests, publishes to GitHub Packages, and generates release notes from Conventional Commits.

Changelog: Must follow Keep a Changelog 2.0.0. Breaking changes are prefixed with `**Breaking : **` in the changelog entry.

See Appendix C for the skill system reference.

11 Verification and Testing

Agent behaviour is validated using a structured evaluation pipeline built on Pydantic AI, per ADR 0007 §11.

Gold datasets:

Versioned Pydantic `BaseModel` schemas define evaluation examples:

Model	Fields	Purpose
<code>GoldExample</code>	<code>input_text</code> , <code>expected_output</code> , <code>difficulty</code> (enum: easy/medium/hard), <code>tags</code> (list)	Single test case
<code>GoldDataset</code>	<code>name</code> , <code>version</code> (SemVer), <code>description</code> , <code>examples</code> (list of <code>GoldExample</code>)	Versioned collection of test cases
<code>VersionedDataset</code>	Tracks SemVer, change description, date	Dataset version metadata

Dataset versioning (SemVer):

Bump	Trigger
Major	Breaking schema changes to <code>GoldExample</code> or evaluation criteria
Minor	New examples added, expanded coverage
Patch	Bug fixes to existing examples (incorrect expected outputs, typos)

LLM-as-judge scoring:

Model	Fields	Purpose
<code>JudgeScore</code>	<code>score</code> (0.0–1.0), <code>reasoning</code> (string), <code>issues</code> (list of strings)	Structured evaluation output
<code>RubricCriterion</code>	<code>name</code> , <code>description</code> , <code>weight</code> (0.0–1.0)	Weighted evaluation dimension
<code>RubricScore</code>	Per-criterion scores with weighted aggregate	Multi-dimensional quality measurement

Rubric criteria include accuracy, completeness, and clarity. Weighted rubrics allow different emphasis per agent role — for example, Developer evaluations weight accuracy higher than clarity, while Tech-Writer evaluations do the reverse.

Init hook testing:

Test Type	Scope
Deterministic unit tests	Version-stamping logic (read, write, compare), file copy (with and without existing files), error handling
Property-based tests	Idempotency (running init hook twice produces identical output), resilience (corrupt stamp file, missing directories, permission errors)

Pipeline integration tests:

Test Type	Scope
End-to-end multi-agent	Full workflow: issue → plan → implement → review → test → deploy
VCR cassette recording	Recorded HTTP interactions (LLM calls) for deterministic replay

Evaluation modes:

Mode	When Used	Purpose
Recorded (VCR)	CI on every PR and push	Deterministic regression detection without live LLM calls
Live	Pre-release final validation	Real-model behaviour confirmation before publishing

See Appendix F for the evaluation and testing reference.

12 Appendix A: OpenCode Plugin System

OpenCode plugins are JavaScript/TypeScript modules that can be loaded from a local directory (`.opencode/plugins/`) or from an npm package (declared via the `"plugin"` key in `opencode.json`). When declared as an npm dependency, the plugin is auto-installed by Bun on OpenCode startup and cached in `~/ .cache/opencode/node_modules/`.

The plugin factory — the exported function from the plugin's entry point — runs on EVERY OpenCode startup, not just on installation. It must therefore be idempotent: running it multiple times must produce the same result. The factory receives a context object containing `project` (name, path, config), `client` (the OpenCode SDK API), `$` (Bun shell), `directory` (current working directory), and `worktree` (git worktree path if applicable).

A critical constraint is that bundled `.opencode/` directories inside an npm package are NOT auto-discovered by OpenCode. Agent and skill files must be explicitly copied into the project's `.opencode/` tree. The recommended pattern is version-stamping: compare a stamp file (containing the installed plugin version) to the actual package version on each startup, and re-copy only on mismatch.

13 Appendix B: Agent System Reference

OpenCode supports three agent modes: `primary` (can be @mentioned and can delegate tasks), `subagent` (can only receive delegated tasks, invisible to the user), and `all` (both capabilities). Agent files use a Markdown format with YAML frontmatter defining `description`, `mode`, `model`, `temperature`, `permission`, and `max_steps`, followed by the system prompt as the body.

Agent discovery is automatic: OpenCode scans `.opencode/agents/` from the current working directory upward to the git worktree root, plus `~/ .config/opencode/agents/`. The filename (without `.md` extension) becomes the agent name, following kebab-case convention.

The permission model uses a three-tier system: `allow` (executes without prompting), `ask` (prompts the user for approval), and `deny` (blocks the action). Bash permissions support per-command granularity via last-match-wins pattern resolution. The `task` permission controls which subagents a primary agent can invoke. The `skill` permission controls which skills an agent can load.

14 Appendix C: Skill System Reference

Skills are lightweight instruction bundles — Markdown files with YAML frontmatter — loaded on-demand via `skill({ name: "..."})`. Unlike agents, skills do not have their own model or permission configuration; they are reusable instruction sets that agents invoke during workflows.

A skill lives in a directory named after the skill, containing a `SKILL.md` file. The frontmatter requires `name` (matching the directory name, regex: `^[a-z0-9]+(-[a-z0-9]+)*$`) and `description` (1–1024 characters). Three trigger types are supported: Manual (invoked by user prompt), Automatic (triggered by webhook), and Chained (invoked by another skill or agent).

The distinction between skills and agents is fundamental: skills are reusable instruction bundles, while agents are full configurations with their own model, permissions, and system prompt. Skills are loaded only on demand, keeping context windows compact.

15 Appendix D: Orchestration Patterns

Three primary orchestration patterns are supported in OpenCode multi-agent systems:

- **Hub-and-Spoke:** A single orchestrator agent delegates tasks to specialist leaf agents. The hub plans work, dispatches via `task()`, collects results, and synthesizes output. Spokes never delegate further. Pros: clear control flow and audit trail. Cons: the architect can become a bottleneck, and conversation context pressure accumulates at the hub.
- **Gated Pipeline:** Sequential stages connected by pass/fail gates with a circuit breaker mechanism (maximum 3 retries before escalation). Each gate enforces a hard contract before the next stage begins, and fail-back routing enables remediation. Pros: quality enforcement and self-correction. Cons: serial execution creates a bottleneck and retry explosion risk.
- **Chain-of-Responsibility:** A request passes through a chain of handlers, each deciding whether to process or forward it. There is no central dispatcher — each node makes an independent judgment. Pros: decoupled and flexible. Cons: hard to trace execution flow and risk of orphaned requests.

RuneSmith uses a combined hub-and-spoke + gated pipeline model: the Architect agent orchestrates work across specialists (hub-and-spoke), while phase gates with circuit breakers enforce quality at each stage (gated pipeline).

16 Appendix E: MCP Reference

MCP (Model Context Protocol) is an open standard for providing external tools to OpenCode agents. It supports two modes: Local, where MCP servers run as subprocesses spawned by OpenCode communicating via stdio; and Remote, where servers are HTTP endpoints using JSON-RPC over POST.

Configuration is done in `opencode.json` under the `mcp` key, specifying a server name, type (local or remote), command, and arguments. MCP servers add tokens to the context window with each tool call, so only necessary servers should be enabled for each agent role.

A current limitation is that the OpenCode plugin SDK does not support programmatic MCP server registration from plugins — all MCP configuration must be done manually in `opencode.json`. If the SDK adds this capability in the future, the init hook can be extended to auto-configure MCP servers from a plugin-defined manifest.

17 Appendix F: Evaluation and Testing Reference

Gold datasets are defined using Pydantic `BaseModel` schemas: `GoldExample` (input_text, expected_output, difficulty as easy/medium/hard, tags), `GoldDataset` (name, version, description, list of examples), and `VersionedDataset` (version tracking with change description and date). Dataset versioning follows SemVer: major bumps for breaking schema changes, minor for new examples, patch for bug fixes.

LLM-as-judge evaluation uses structured scoring via `JudgeScore` (score 0.0–1.0, reasoning, list of issues) and `RubricScore` with weighted `RubricCriterion` objects for multi-dimensional evaluation. Rubric criteria include accuracy, completeness, and clarity, with configurable weights per agent role — for example, Developer evaluations weight accuracy higher than clarity, while Tech-Writer evaluations do the reverse.

Two evaluation modes are supported: Recorded (using VCR cassettes for deterministic replay in CI without live LLM calls) and Live (real model validation before release). This ensures reliable regression testing without incurring LLM costs on every CI run, while still validating against real models prior to publishing.

18 Changelog

18.1 [1.0.0] - 2026-06-14

18.1.1 Added

- Initial proposal document set covering the full [@runicegines/opencode-runesmith](#) implementation plan
- 11 sections: Motivation, Plugin Package Structure, Init Hook & Version-Stamping, Agent Architecture, Skill Architecture, KB Discovery, MCP Integration, Permission Model, Rollout Strategy, Maintenance & Governance, Verification & Testing
- 6 appendices: OpenCode Plugin System, Agent System Reference, Skill System Reference, Orchestration Patterns, MCP Reference, Evaluation & Testing Reference